SHADEWATER LABS

Skill Explainer

Webp Me Daddy

A reusable Codex skill for turning website images into recipe-driven, lintable WebP assets with structured metadata, accessibility-safe alt text, framing-aware crops, and paste-ready snippets.

Shave bytes. Keep vibes. Retire hero-final-final.jpg with dignity.

Semantic recipes	**Review heroes**	**Framing intent**	**Contain logos**
Usage overrides	**Audit reports**	**Cleanup mode**	**Animated handoff**

What The Skill Does

The skill is built for the moment where a raw source image needs to become a real production asset: cropped for the slot it will live in, named cleanly, described accessibly, and returned with snippets that are ready to paste.

- Convert **.jpg** and **.png** files into optimized **.webp** assets.
- Use semantic recipes such as **hero-banner**, **review-hero**, **blog-cover**, **profile-avatar**, **poster**, **story-cover**, **logo-lockup**, and **logo-grid**.
- Guide cover crops with framing intent like **subject**, **text**, or **balanced** before dropping to exact focus coordinates.
- Preserve transparent logos and lockups with contain-mode outputs instead of forcing a crop.
- Generate SEO-friendly slugs, suggested **alt**, **title**, and **caption** metadata, plus usage-specific overrides for shared assets.
- Emit paste-ready **HTML**, **React**, **Next.js**, and **Astro** snippets inside a stable sidecar and manifest contract.
- Audit a real codebase for live legacy formats, animated assets, stale public files, and markup gaps, then suggest or apply low-risk fixes.
- Hand animated GIF and WebP inputs to the transparent loop optimizer from the same top-level CLI.

How The Workflow Thinks

The strongest part of Webp Me Daddy is that it starts from placement intent instead of raw image math. That keeps the workflow useful for solo creators, developers, and future app interfaces without constantly asking the user to think in crop ratios first.

Recipes describe the job

Pick a layout role like hero-banner, review-hero, or logo-lockup so widths, crop shape, and loading defaults match the real placement.

Framing describes the priority

Use subject, text, or balanced framing when a crop needs to preserve a face, a title, or a compromise between the two.

Accessibility and usage stay explicit

Decorative, logo, descriptive, and text-bearing modes keep alt behavior deliberate, while usage overrides let one asset stay honest across homepage, deck, and article placements.

Audit closes the loop

The same skill can review a codebase, surface stale assets, suggest safe fixes, clean out leftovers, and hand animated files to the loop workflow instead of forcing them through still-image prep.

Typical Flow

In practice the workflow compresses into four repeatable moves: choose the role, frame it intentionally, generate the reusable output, and review the result before it ships.

STEP 1

Choose the role

Pick the recipe and accessibility mode based on where the image will live and whether it is informative, decorative, logo-like, or text-bearing.

STEP 2

Frame it on purpose

Use framing intent first, then refine with focus presets or exact coordinates only if the default crop still misses the mark.

STEP 3

Generate the reusable output

Prepare the WebP, optional responsive variants, and the structured sidecar or manifest entry that keeps the result explainable.

STEP 4

Review before shipping

Generate the right snippet, run lint or audit when needed, clean stale sources, and hand animated files off to the loop workflow instead of forcing them through still-image prep.

Where It Helps Most

The skill is most useful when you want opinionated defaults that still leave room for editorial judgment. It does best in projects where image quality, readability, and code integration all matter at once.

- Turn a homepage image into a proper **hero-banner** with responsive variants and eager-loading defaults.
- Transform movie posters or game key art into **review-hero** crops that match notes cards and full review headers cleanly.
- Prepare creator portraits as **profile-avatar** assets with cleaner naming and safer sizes.
- Preserve sponsor badges, lockups, and partner logos with **logo-lockup** or **logo-grid** recipes instead of cropping them into awkward shapes.
- Run a batch preview before touching a large public folder, then lint or audit the results before a deployment.
- Use usage overrides when the same image appears in a homepage hero, a sponsor deck, and a post header with different alt needs.
- Audit a mature codebase to find lingering PNG and JPEG usage, shared-context assets, animated files, unused originals, or missing width and loading hints.
- Regenerate page-specific snippets from the sidecar after a copy edit without reprocessing the source image.

What You Get Back

Optimized assets

A main WebP plus optional responsive variants like **image-480w.webp** or **image-960w.webp**.

Structured metadata

Per-image sidecars and run-level manifests that store recipe, framing, dimensions, bytes, usage overrides, and analysis fields.

Paste-ready snippets

HTML, React, Next.js, and Astro markup with escaped attributes and the right public paths for the site build.

Project-level reports

Audit and cleanup JSON output for codebase reviews, autofix suggestions, codemod patches, apply-mode results, and reclaimed bytes.

Guardrails That Matter

- Small source images stay at native size by default so recipe targets do not accidentally upscale them.
- Poster-to-landscape review crops bias toward the main subject unless the user explicitly says title text must stay visible.
- Contain-mode outputs protect transparent edges and lockups that should never be trimmed just to satisfy a crop target.
- Audit apply mode stays conservative and only writes exact, single-match replacements for low-risk image-tag fixes.

- Framework-aware audit suggestions preserve **next/image** semantics instead of flattening everything into raw HTML assumptions.

Command Patterns

These are the commands that matter most in day-to-day use. They are short enough to copy, but opinionated enough to show the intended shape of the workflow.

Prepare one review hero

```
python C:/Users/Alex/.codex/skills/webp-me-daddy/scripts/webp_me_daddy.py prepare
public/poster.jpg
--recipe review-hero --framing subject --public-root public --write-sidecar
```

Prepare a transparent logo lockup

```
python C:/Users/Alex/.codex/skills/webp-me-daddy/scripts/webp_me_daddy.py prepare
public/logo.png
--recipe logo-lockup --accessibility-mode logo --public-root public --write-sidecar
```

Preview a batch run

```
python C:/Users/Alex/.codex/skills/webp-me-daddy/scripts/webp_me_daddy.py batch public
--recipe blog-cover --dry-run --manifest image-manifest.json
```

Audit and apply safe fixes

```
python C:/Users/Alex/.codex/skills/webp-me-daddy/scripts/webp_me_daddy.py audit
C:/path/to/project
--apply-autofix --json image-audit.json
```

Rebuild page-specific markup

```
python C:/Users/Alex/.codex/skills/webp-me-daddy/scripts/webp_me_daddy.py snippets
public/hero-source.json
--usage-key home.hero --target react
```

Hand off an animated asset

```
python C:/Users/Alex/.codex/skills/webp-me-daddy/scripts/webp_me_daddy.py animate
public/spin.webp public/spin-clean.webp
--size 220 --bridge-frames 8
```

Validation

- Run **test_prepare_image.py** after meaningful script changes.
- Regenerate the explainer PDF and visually review the latest pages instead of trusting the source alone.

- Smoke-test **audit**, **cleanup --dry-run**, **snippets**, and **animate** against a real project so the helper flows stay honest.
- Run the skill validator before shipping documentation updates or command changes.

> The future web app should inherit this same structure: recipe-first inputs, framing-aware crops, structured sidecars, and audit-style reporting, rather than becoming a generic upload-and-download converter.